Knowledge Representation for Defect Handling in Call Testing

AIMLZG557 Artificial and Computational Intelligence | Situated Learning Assignment | Question 2 (Individual) | Section B: Student Submission

B.1 Submission Form

Name	Neelam Devi
ID No	2025AG05908
Industry / Domain	Consumer electronics: mobile phone software testing (call testing)
Organisation Type	Large multinational mobile phone manufacturer
[Instructor prompt]	Identify a real workplace process where decisions are governed by predefined rules.

B.2 Anchor (1 mark)

Workplace artifact: the defect handling process on our defect management portal, from the day a tester reports a defect till the project gets its readiness approval. I work as a tester in call testing of mobile phones and I have 14 years of experience in mobile testing. Every step of this process follows rules given by the project leader (PL) and the model owner (MO).

Why it matters. A phone reaches many users, so a critical defect like a call force close must not go to the market. But late in the project every code change can bring side effects and new issues, so not every defect can be fixed in every phase. That is why the rules decide which defects are reported in which phase, and every tester and developer has to follow the same rules.

How a defect flows:

1. The tester finds an issue and reports it on the portal with logs, reproduction steps, reproduction rate, screenshot or video, severity and the module. A defect is reported even if it comes only sometimes (for example 5 times out of 10), because the issue has come to the tester.
2. The MO reviews it. If there is a mistake he tells the tester, the tester corrects it, he reviews again and then submits it under the tester's name.
3. After submission the portal assigns the defect to the R&D developer of the module selected by the tester, so selecting the right module matters. The developer writes the cause and the countermeasure and marks a code fix, a concept issue or a limitation. If the issue belongs to another module, for example audio, he assigns it to that module's developer and the same thing continues in the comments of the same defect.
4. For a code fix the developer adds the patch and the binary (software) in which it will come. Till that binary is released the defect stays in not release state. The project manager collects the fixes of all submitted builds, releases a new software and the defect then shows as release.
5. The tester flashes the new software and checks the same scenario. If the issue is not seen, the tester closes the defect within 24 hours so that the turnaround time does not increase. If it is still seen, the tester rejects it and the developer analyses again.
6. A concept issue or a limitation needs no new software. The developer resolves it on the same reported software and the tester closes it. If the issue cannot be reproduced, it is marked irreproducible and closed on the same binary.
7. If the issue is in a third party app, it is reported to the app vendor. There is no new phone software, the fix comes as a new apk version through the Play Store or the Samsung store.
8. When all test cases are done and passed (a failed test item is tested again till it passes) and no critical defect is open, the MO gives the project readiness approval, which means testing from our side is complete and the software is ready for production.

What decides if a defect is reported. Severity is decided by the tester: A is a critical issue like phone lockup or force close which comes always, B is a function that does not work but the phone can still be used, and C is a display issue like crop or overlap. In DEV phase (start of the project, first binaries) all issues are reported, even if seen only once. In PR phase (final binaries stage) only critical defects, or functionality which blocks daily phone use (for example a call is accepted but needs two swipes instead of one), are reported. Low impact B defects and C defects are not reported then, so that there is less code change and no side effects. At the final binary only A priority defects are reported. In MR phase the model is already in the market and known low impact defects and market feedback (VOC) are retested and handled. If the project has to finish early, some low impact defects are also moved to MR phase. Which severities are reported in which phase, and which defects can move to MR phase and in what percentage, is decided in the sync-up of the PL, the MO and the project manager.

Who is involved: the tester (tests properly and reports the maximum defects as per the phase), the test manager (reviews and manages the defects with the MO), the MO (shares the rules, gets testing done on time and gives the readiness approval), the PL (coordinates testing and R&D on timeline), the developers (fix on time with good analysis) and the project manager (releases the software).

Where the facts come from: the defect portal (severity, module, logs, status, developer outcome and comments), the test results of the project, the sync-up decisions of the PL and the MO, and the software release information of the project manager.

Why a rule-based knowledge base fits: the phase rules and the defect steps are already if-then rules, every defect should get the same decision each time, and today the reason for a decision is scattered in the comments of the defect. A knowledge base can show the rules and facts behind each answer.

Defect IDs, project names and data in this document are made up. The rules are a simplified form of the guidelines we follow.

B.3 Apply (2 marks)

The process gives 18 business rules (B.3.1). They are written in propositional logic (B.3.2), extended to predicate logic (B.3.3), turned into a knowledge base of 61 facts and 18 inference rules (B.3.4), and used to answer five workplace queries (B.3.5).

B.3.1 Business rules

Rule	Business rule
BR1	In DEV phase every defect can be reported, even if it is seen only once.
BR2	In PR phase a defect is reported if its severity is allowed in the PL and MO sync-up (normally A), even if it is seen only once.
BR3	In PR phase a defect that blocks daily phone use is reported, even if it comes only sometimes (for example 5 times out of 10).
BR4	In PR phase a defect that is not reportable by BR2 or BR3 is not reported now, it goes to MR phase.
BR5	If a reportable defect has no logs, the MO returns it to the tester.
BR6	If a reportable defect has the wrong module selected, the MO returns it to the tester.
BR7	If a reportable defect has logs and the right module, the MO submits it.
BR8	A submitted defect which is not a third party issue is assigned to the developer of the selected module.
BR9	A submitted third party app defect is reported to the app vendor and not to R&D.
BR10	If the developer marks a code fix and the binary is not released, the defect stays in not release state.
BR11	If the developer marks a code fix and the binary is released, the defect is in release state.
BR12	If the developer marks concept or limitation, or the issue is irreproducible, it is resolved on the same binary and no new software is needed.
BR13	A defect resolved on the same binary is closed by the tester.
BR14	A defect in release state which is not seen on the new software is closed by the tester within 24 hours.

Rule	Business rule
BR15	A defect in release state which is still seen on the new software is rejected and goes back to the developer.
BR16	A submitted severity A defect which is not closed blocks its project.
BR17	If any defect blocks a project, the project is not ready.
BR18	If all test cases are done and passed, and no defect blocks the project, the project is ready for the MO's readiness approval.

B.3.2 Propositional logic

Each proposition is a statement about the one defect (or the one project) under review, and it is either true or false. "Blocks daily use" means the problem directly affects normal phone use, for example a call that needs two swipes to accept instead of one.

Symbol	Meaning	Symbol	Meaning
DevPhase	Project is in DEV phase	BinaryReleased	Fix binary is released
PRPhase	Project is in PR phase	NotReleased	Defect is in not release state
SevA	Severity is A	Released	Defect is in release state
AllowedBySyncUp	Severity allowed in sync-up for PR	Concept	Developer marked concept issue
BlocksDailyUse	Defect blocks daily phone use	Limitation	Developer marked limitation
Reportable	Defect should be reported now	Irreproducible	Issue marked irreproducible
DeferMR	Defect goes to MR phase	ResolveSameBinary	Resolved on the same binary
HasLogs	Logs are attached	NotReproducedOnNew	Not seen on new software
WrongModule	Wrong module selected	ReproducedOnNew	Still seen on new software
ReturnedByMO	MO returns it to tester	Closed	Defect is closed
Submitted	MO has submitted the defect	Rejected	Rejected back to developer
ThirdParty	Issue is in a third party app	BlocksReadiness	Defect blocks its project
AssignedToRD	Assigned to module developer	NotReady	Project is not ready
ReportedToVendor	Reported to app vendor	AllTestsDone	All test cases done and passed
CodeFix	Developer marked a code fix	ReadyForApproval	Project ready for MO approval

The 18 business rules in propositional logic:

Rule	Propositional formula
BR1	DevPhase → Reportable
BR2	PRPhase ∧ AllowedBySyncUp → Reportable
BR3	PRPhase ∧ BlocksDailyUse → Reportable
BR4	PRPhase ∧ ¬Reportable → DeferMR
BR5	Reportable ∧ ¬HasLogs → ReturnedByMO
BR6	Reportable ∧ WrongModule → ReturnedByMO
BR7	Reportable ∧ HasLogs ∧ ¬WrongModule → Submitted
BR8	Submitted ∧ ¬ThirdParty → AssignedToRD
BR9	Submitted ∧ ThirdParty → ReportedToVendor
BR10	Submitted ∧ CodeFix ∧ ¬BinaryReleased → NotReleased
BR11	Submitted ∧ CodeFix ∧ BinaryReleased → Released
BR12	Submitted ∧ (Concept ∨ Limitation ∨ Irreproducible) → ResolveSameBinary
BR13	ResolveSameBinary → Closed
BR14	Released ∧ NotReproducedOnNew → Closed
BR15	Released ∧ ReproducedOnNew → Rejected

Rule	Propositional formula
BR16	Submitted ∧ SevA ∧ ¬Closed → BlocksReadiness
BR17	BlocksReadiness → NotReady
BR18	AllTestsDone ∧ ¬BlocksReadiness → ReadyForApproval

In propositional logic every symbol is about one single defect. For nine defects I would need nine copies of every symbol and rule, like SevA_D1, SevA_D2 and so on. BR16 to BR18 talk about any defect of a project, and propositional logic cannot say which project a defect belongs to or which developer owns which module. So the same rules are written again in predicate logic.

B.3.3 Predicate logic

The same rules extended with constants, variables, predicates and quantifiers.

Constants

Type	Constants
Defects	D1, D2, D3, D4, D5, D6, D7, D8, D9
Projects	M1, M2
Severity	A, B, C
Phases	Dev, PR
Modules	Call, Audio
Developers	DevCall, DevAudio
Developer outcomes	Code, Concept, Limitation, Irreproducible
Reproduction rate	5of10

Variables: d a defect, p a project, s a severity, m a module, v a developer, o an outcome.
Quantifiers: ∀ (for all), ∃ (there exists), ¬∃ (there is none).

Predicates

Predicate	Meaning	Kind
FoundIn(d, p)	Defect d was found in project p	Fact
Phase(p, x)	Project p is in phase x	Fact
Sev(d, s)	Defect d has severity s	Fact
SyncUpAllows(p, s)	Sync-up allows severity s in PR phase for project p	Fact
BlocksDailyUse(d)	Defect d blocks daily phone use	Fact
HasLogs(d)	Logs are attached to d	Fact
WrongModule(d)	Wrong module is selected for d	Fact
Module(d, m)	Module m is selected for d	Fact
DeveloperOf(m, v)	Developer v owns module m	Fact
ThirdParty(d)	Defect d is a third party app issue	Fact
Outcome(d, o)	Developer outcome o is marked on d	Fact
NoCodeChange(o)	Outcome o needs no new software	Fact
BinaryReleased(d)	The fix binary of d is released	Fact
NotReproducedOnNew(d)	d is not seen on the new software	Fact
ReproducedOnNew(d)	d is still seen on the new software	Fact
Project(p)	p is a project	Fact
AllTestsDone(p)	All test cases of p are done and passed	Fact
SeenOnce(d)	Defect d was seen only once	Fact
ReproRate(d, r)	Defect d was seen r times, for example 5of10	Fact

Predicate	Meaning	Kind
Reportable(d)	d should be reported now	Derived
DeferMR(d)	d goes to MR phase	Derived
Returned(d)	The MO returned d to the tester	Derived
Submitted(d)	The MO submitted d	Derived
AssignedTo(d, v)	d is assigned to developer v	Derived
ReportToVendor(d)	d goes to the app vendor	Derived
NotRelease(d)	d is in not release state	Derived
Release(d)	d is in release state	Derived
ResolveSameBinary(d)	d is resolved on the same binary	Derived
Closed(d)	d is closed	Derived
Rejected(d)	d is rejected back to the developer	Derived
BlocksReadiness(d, p)	Defect d blocks project p	Derived
NotReady(p)	Project p is not ready	Derived
ReadyForApproval(p)	Project p can get the MO approval	Derived

The 18 business rules in predicate logic:

Rule	Predicate logic formula
BR1	∀d ∀p [FoundIn(d, p) ∧ Phase(p, Dev) → Reportable(d)]
BR2	∀d ∀p ∀s [FoundIn(d, p) ∧ Phase(p, PR) ∧ Sev(d, s) ∧ SyncUpAllows(p, s) → Reportable(d)]
BR3	∀d ∀p [FoundIn(d, p) ∧ Phase(p, PR) ∧ BlocksDailyUse(d) → Reportable(d)]
BR4	∀d ∀p [FoundIn(d, p) ∧ Phase(p, PR) ∧ ¬Reportable(d) → DeferMR(d)]
BR5	∀d [Reportable(d) ∧ ¬HasLogs(d) → Returned(d)]
BR6	∀d [Reportable(d) ∧ WrongModule(d) → Returned(d)]
BR7	∀d [Reportable(d) ∧ HasLogs(d) ∧ ¬WrongModule(d) → Submitted(d)]
BR8	∀d ∀m ∀v [Submitted(d) ∧ Module(d, m) ∧ DeveloperOf(m, v) ∧ ¬ThirdParty(d) → AssignedTo(d, v)]
BR9	∀d [Submitted(d) ∧ ThirdParty(d) → ReportToVendor(d)]
BR10	∀d [Submitted(d) ∧ Outcome(d, Code) ∧ ¬BinaryReleased(d) → NotRelease(d)]
BR11	∀d [Submitted(d) ∧ Outcome(d, Code) ∧ BinaryReleased(d) → Release(d)]
BR12	∀d ∀o [Submitted(d) ∧ Outcome(d, o) ∧ NoCodeChange(o) → ResolveSameBinary(d)]
BR13	∀d [ResolveSameBinary(d) → Closed(d)]
BR14	∀d [Release(d) ∧ NotReproducedOnNew(d) → Closed(d)]
BR15	∀d [Release(d) ∧ ReproducedOnNew(d) → Rejected(d)]
BR16	∀d ∀p [Submitted(d) ∧ Sev(d, A) ∧ FoundIn(d, p) ∧ ¬Closed(d) → BlocksReadiness(d, p)]
BR17	∀p [Project(p) ∧ ∃d BlocksReadiness(d, p) → NotReady(p)]
BR18	∀p [Project(p) ∧ AllTestsDone(p) ∧ ¬∃d BlocksReadiness(d, p) → ReadyForApproval(p)]

How the quantifiers are used:

1. ∀ over d and p makes one rule cover every defect and every project.
2. BR2 also uses ∀s: the severity allowed in the sync-up is a variable, so SyncUpAllows(p, s) can be changed for each project.
3. BR8 uses ∀m ∀v: the developer of the selected module is found with DeveloperOf(m, v), which is what the portal does when the tester selects a module.
4. BR12 uses ∀o: NoCodeChange(o) covers concept, limitation and irreproducible together.
5. ∃d in BR17 means "some defect blocks the project" and ¬∃d in BR18 means "no defect blocks the project".

6. A not-condition like ¬HasLogs(d) is true when the KB has no such fact (closed world assumption, see B.3.4).

B.3.4 Knowledge base

The knowledge base holds **61 facts** and **18 inference rules**. I also coded it in Python (Appendix A) and ran it, so every answer in B.3.5 was checked by the program.

Defects in the knowledge base

Defect	What it is	Project (phase)	Sev	Module	Position in the process
D1	Dialer force close on an incoming call	M1 (PR)	A	Call	Developer marked code fix, binary not released
D2	Incoming call needs two swipes to accept	M1 (PR)	B	Call	Code fix, binary released, not seen on new software
D3	Text overlap on the in-call screen	M1 (PR)	C	Call	Nothing marked yet
D4	One way audio, seen only once	M1 (PR)	A	Audio	Wrong module selected
D5	Call drops on network switch, seen 5 times out of 10	M1 (PR)	B	Call	Logs not attached
D6	Hold icon cropped, seen only once	M2 (DEV)	C	Call	Developer marked concept
D7	Crash inside a third party calling app	M2 (DEV)	B	None	Third party app issue
D8	Call recording fails	M2 (DEV)	B	Call	Code fix, binary released, still seen
D9	Call mutes after hold	M2 (DEV)	B	Call	Developer could not reproduce it

Facts (61), in groups

Facts	Group and facts	Count
F1-F11	**Setup:** Project(M1), Phase(M1, PR), AllTestsDone(M1), SyncUpAllows(M1, A), Project(M2), Phase(M2, Dev), DeveloperOf(Call, DevCall), DeveloperOf(Audio, DevAudio), NoCodeChange(Concept), NoCodeChange(Limitation), NoCodeChange(Irreproducible)	11
F12-F16	**D1:** Sev(D1, A), HasLogs(D1), FoundIn(D1, M1), Module(D1, Call), Outcome(D1, Code)	5
F17-F24	**D2:** Sev(D2, B), BlocksDailyUse(D2), HasLogs(D2), FoundIn(D2, M1), Module(D2, Call), Outcome(D2, Code), BinaryReleased(D2), NotReproducedOnNew(D2)	8
F25-F28	**D3:** Sev(D3, C), HasLogs(D3), FoundIn(D3, M1), Module(D3, Call)	4
F29-F34	**D4:** Sev(D4, A), SeenOnce(D4), HasLogs(D4), FoundIn(D4, M1), Module(D4, Audio), WrongModule(D4)	6
F35-F39	**D5:** Sev(D5, B), BlocksDailyUse(D5), ReproRate(D5, 5of10), FoundIn(D5, M1), Module(D5, Call)	5
F40-F45	**D6:** Sev(D6, C), SeenOnce(D6), HasLogs(D6), FoundIn(D6, M2), Module(D6, Call), Outcome(D6, Concept)	6
F46-F49	**D7:** Sev(D7, B), HasLogs(D7), FoundIn(D7, M2), ThirdParty(D7)	4
F50-F56	**D8:** Sev(D8, B), HasLogs(D8), FoundIn(D8, M2), Module(D8, Call), Outcome(D8, Code), BinaryReleased(D8), ReproducedOnNew(D8)	7
F57-F61	**D9:** Sev(D9, B), HasLogs(D9), FoundIn(D9, M2), Module(D9, Call), Outcome(D9, Irreproducible)	5
	Total	**61**

Only positive facts are stored. A fact which is not in the KB and cannot be derived is taken as false (closed world assumption). For example D5 has no HasLogs fact, so D5 has no logs. SeenOnce and ReproRate are kept only as information, no rule uses them, because a defect is reported even if it is seen once or only sometimes.

Inference rules (18). Rn is the business rule BRn written for the KB. The variables are universally quantified and each rule has one conclusion.

Rule	Inference rule	From	Stage
R1	FoundIn(d, p) ∧ Phase(p, Dev) → Reportable(d)	BR1	1
R2	FoundIn(d, p) ∧ Phase(p, PR) ∧ Sev(d, s) ∧ SyncUpAllows(p, s) → Reportable(d)	BR2	1
R3	FoundIn(d, p) ∧ Phase(p, PR) ∧ BlocksDailyUse(d) → Reportable(d)	BR3	1
R4	FoundIn(d, p) ∧ Phase(p, PR) ∧ ¬Reportable(d) → DeferMR(d)	BR4	2
R5	Reportable(d) ∧ ¬HasLogs(d) → Returned(d)	BR5	1
R6	Reportable(d) ∧ WrongModule(d) → Returned(d)	BR6	1
R7	Reportable(d) ∧ HasLogs(d) ∧ ¬WrongModule(d) → Submitted(d)	BR7	1
R8	Submitted(d) ∧ Module(d, m) ∧ DeveloperOf(m, v) ∧ ¬ThirdParty(d) → AssignedTo(d, v)	BR8	1
R9	Submitted(d) ∧ ThirdParty(d) → ReportToVendor(d)	BR9	1
R10	Submitted(d) ∧ Outcome(d, Code) ∧ ¬BinaryReleased(d) → NotRelease(d)	BR10	1
R11	Submitted(d) ∧ Outcome(d, Code) ∧ BinaryReleased(d) → Release(d)	BR11	1
R12	Submitted(d) ∧ Outcome(d, o) ∧ NoCodeChange(o) → ResolveSameBinary(d)	BR12	1
R13	ResolveSameBinary(d) → Closed(d)	BR13	1
R14	Release(d) ∧ NotReproducedOnNew(d) → Closed(d)	BR14	1
R15	Release(d) ∧ ReproducedOnNew(d) → Rejected(d)	BR15	1
R16	Submitted(d) ∧ Sev(d, A) ∧ FoundIn(d, p) ∧ ¬Closed(d) → BlocksReadiness(d, p)	BR16	2
R17	Project(p) ∧ BlocksReadiness(d, p) → NotReady(p)	BR17	3
R18	Project(p) ∧ AllTestsDone(p) ∧ ¬∃d BlocksReadiness(d, p) → ReadyForApproval(p)	BR18	3

The not-conditions are checked only after the fact they depend on is complete. So the rules are applied in three stages: stage 1 for the rules without a late not-condition, stage 2 for R4 (needs the final Reportable) and R16 (needs the final Closed), and stage 3 for R17 and R18 (need the final BlocksReadiness).

B.3.5 Workplace queries

These are questions our team asks about the defects of the two projects. Forward chaining starts from the facts and applies rules till the answer appears (modus ponens: from P and P → Q conclude Q, with the variables replaced by the actual defect, project or developer). Backward chaining starts from the goal and works back to the facts. KB ⊢ Q means that Q can be derived from the knowledge base.

Query 1: D3 is a display overlap (severity C) in project M1, which is in PR phase. Should it be reported now?

Method: forward chaining, with a not-condition.

Step	New fact	Rule	Because of
1	Reportable(D3) is not derived	R1	needs Phase(M1, Dev), but M1 is in PR phase
2	Reportable(D3) is not derived	R2	needs SyncUpAllows(M1, C), the sync-up allows only A for M1
3	Reportable(D3) is not derived	R3	needs BlocksDailyUse(D3), which is not in the KB because D3 is only a display issue
4	DeferMR(D3)	R4	FoundIn(D3, M1), Phase(M1, PR), and Reportable(D3) is not derived

Answer: KB ⊢ DeferMR(D3). D3 is not reported now, it goes to MR phase.

Query 2: What happens to D1, the dialer force close (severity A)?

Method: forward chaining.

Step	New fact	Rule	Because of
1	Reportable(D1)	R2	FoundIn(D1, M1), Phase(M1, PR), Sev(D1, A), SyncUpAllows(M1, A)
2	Submitted(D1)	R7	Reportable(D1), HasLogs(D1), and WrongModule(D1) is not in the KB
3	AssignedTo(D1, DevCall)	R8	Submitted(D1), Module(D1, Call), DeveloperOf(Call, DevCall), and ThirdParty(D1) is not in the KB
4	NotRelease(D1)	R10	Submitted(D1), Outcome(D1, Code), and BinaryReleased(D1) is not in the KB

Answer: KB ⊢ AssignedTo(D1, DevCall) and KB ⊢ NotRelease(D1). D1 is submitted, goes to the call developer and stays in not release state till its binary is released.

Query 3: D4 is one way audio (severity A, seen only once) and the module selected is wrong. Does it reach the developer?

Method: forward chaining, where a not-condition stops a rule.

Step	New fact	Rule	Because of
1	Reportable(D4)	R2	FoundIn(D4, M1), Phase(M1, PR), Sev(D4, A), SyncUpAllows(M1, A). SeenOnce(D4) does not matter for severity A.
2	Returned(D4)	R6	Reportable(D4), WrongModule(D4)
3	Submitted(D4) is not derived	R7	needs not WrongModule(D4), but WrongModule(D4) is a fact. So R8 also cannot fire.

Answer: KB ⊢ Returned(D4). D4 is reportable, but the MO returns it to the tester, so it does not reach the developer. After the tester corrects the module, WrongModule(D4) is no more true and R7 and R8 submit it and assign it.

Query 4: New software is released. What happens to D2 and D8?

Method: backward chaining for D2, forward chaining for D8.

D2: start from the goal Closed(D2) and work back to the facts.

Step	Goal	How it is resolved	Result
1	Closed(D2)	R13 and R14 conclude Closed. Try R13 first.	2 rules
2	ResolveSameBinary(D2) by R13	R12 needs NoCodeChange(o) for the outcome of D2. Outcome(D2, Code) is a fact, but NoCodeChange(Code) is not.	fails
3	Closed(D2) by R14	Needs Release(D2) and NotReproducedOnNew(D2).	2 subgoals
4	Release(D2)	R11 needs Submitted(D2), Outcome(D2, Code) and BinaryReleased(D2). The last two are facts.	1 subgoal left
5	Submitted(D2)	R7 needs Reportable(D2), HasLogs(D2) and no WrongModule(D2). HasLogs is a fact and WrongModule is not.	1 subgoal left
6	Reportable(D2)	R3: FoundIn(D2, M1), Phase(M1, PR) and BlocksDailyUse(D2) are facts. (R2 does not work because SyncUpAllows(M1, B) is not a fact.)	proved
7	NotReproducedOnNew(D2)	It is a fact in the KB.	proved
8	Closed(D2)	All subgoals of R14 are proved.	proved

D8: forward chaining.

Step	New fact	Rule	Because of
1	Reportable(D8)	R1	FoundIn(D8, M2), Phase(M2, Dev)
2	Submitted(D8)	R7	Reportable(D8), HasLogs(D8), and WrongModule(D8) is not in the KB
3	Release(D8)	R11	Submitted(D8), Outcome(D8, Code), BinaryReleased(D8)
4	Rejected(D8)	R15	Release(D8), ReproducedOnNew(D8)

Answer: KB ⊢ Closed(D2), so the tester closes D2 within 24 hours. KB ⊢ Rejected(D8), so D8 is rejected and goes back to the developer.

Query 5: Can the MO give readiness approval for project M1?

Method: forward chaining with ∃ and ¬∃, then a what-if case.

Step	New fact	Rule	Because of
1	BlocksReadiness(D1, M1)	R16	Submitted(D1), Sev(D1, A), FoundIn(D1, M1), and Closed(D1) is not derived
2	NotReady(M1)	R17	Project(M1), BlocksReadiness(D1, M1)
3	ReadyForApproval(M1) is not derived	R18	AllTestsDone(M1) is a fact, but R18 needs that no defect blocks M1, and D1 blocks it

Answer: KB ⊢ NotReady(M1). No, M1 is not ready because D1 is still open.

What if the fixed binary of D1 is released and D1 is not seen on it?

Step	New fact	Rule	Because of
1	Release(D1)	R11	Submitted(D1), Outcome(D1, Code), BinaryReleased(D1)
2	Closed(D1)	R14	Release(D1), NotReproducedOnNew(D1)
3	ReadyForApproval(M1)	R18	Project(M1), AllTestsDone(M1), and no BlocksReadiness(d, M1) because R16 does not fire for the closed D1

Answer: KB ⊢ ReadyForApproval(M1) in the what-if case, so the MO can give the approval.

B.4 Analyse (1 mark)

How predicate logic improves on propositional logic here

Predicate logic lets one set of 18 rules decide every defect and project. In propositional logic every symbol is about one defect, so 18 rules for 9 defects can need up to 18 x 9 = 162 rule copies, and a real project has hundreds of defects.

What the workplace needs	Propositional logic	Predicate logic
Many defects and projects	Symbols describe one defect. Nine defects need nine copies of the symbols and rules, and every new defect needs new symbols.	One rule covers all defects through the variables d and p. A new defect is just new facts. The same 18 rules answered all five queries.
Phase based reporting (BR1 to BR4)	DevPhase and PRPhase can only be asserted for the one defect. The link between a defect and the phase of its project is missing.	FoundIn(d, p) and Phase(p, PR) link each defect to its project. D1 in M1 (PR) and D8 in M2 (DEV) are handled by the same rules.
Right developer for a module (BR8)	AssignedToRD is only true or false. It cannot say which developer.	Module(d, m) and DeveloperOf(m, v) find the developer. Query 2 gave DevCall for D1.
Severity allowed in sync-up (BR2)	AllowedBySyncUp is simply asserted for the one defect.	Sev(d, s) and SyncUpAllows(p, s) compare the severity of the defect with the project setting. A new sync-up decision is one new fact.
"Some defect blocks the project" (BR16 to BR18)	BlocksReadiness can be written for one defect only, so "any" or "none" cannot be said.	∃d and ¬∃d say it. Query 5 found D1 as the blocker of M1.
Open questions	Only yes or no about one fixed defect.	A query with a variable returns the answers. For example Submitted(d) gives D1, D2, D6, D7, D8 and D9 (see the derived facts in Appendix A).

How well the knowledge base solves the problem

The knowledge base answered all five queries and the other defects the way the process works, and every answer comes with the chain of rules and facts behind it. To check it, I wrote what I expect by hand for every defect and for project M1 and compared it with the KB result:

Item	What I expect by hand	KB result (rules used)	Same
D1	Reported, submitted, goes to the call developer, waits for release	R2, R7, R8, R10	Yes
D2	Reported, closed after the new software	R3, R7, R11, R14	Yes
D3	Not reported in PR phase, kept for MR	R4	Yes
D4	Reported, MO returns it for wrong module	R2, R6	Yes
D5	Reported, MO returns it for missing logs	R3, R5	Yes
D6	Reported in DEV, concept, closed on same binary	R1, R7, R12, R13	Yes
D7	Third party app, goes to the vendor	R1, R7, R9	Yes
D8	Still seen on new software, so rejected	R1, R7, R11, R15	Yes
D9	Irreproducible, closed on same binary	R1, R7, R12, R13	Yes
Project M1	Not ready, because D1 is open	R2, R7, R16, R17	Yes

1. It gives the same decision every time, whoever asks. For the same facts the phase rules always give the same answer.
2. The reason is visible. Today the reasons are scattered in the comments of a defect, here the rules and facts behind an answer are listed.
3. It catches things that are easy to miss: a defect returned for a wrong module or missing logs, and a critical defect which is still open when the project is going for approval.

Missing knowledge and limitations

1. **Judgement is taken as input.** Severity, "blocks daily use" and the developer's outcome are decisions of people, and the KB takes them as given. If somebody is wrong, the KB repeats the mistake. So the severity differences and the wrong concept marking are not solved by it.
2. **No sense of time.** The 24 hour closing, the late retest of failed test items and the turnaround time cannot be checked.
3. **Sync-up decisions are only facts.** Which severities are reported in each phase, and which defects can go to MR phase and in what percentage, are stored as facts like SyncUpAllows(M1, A). If nobody updates them after a new meeting, the reporting rules give wrong answers. The MR percentage is not modelled.
4. **Returned defects do not block readiness.** A defect which the MO has returned (like D4) does not count in R16, although it is a critical defect.
5. **Retest is not modelled.** The retest of a failed test item till it passes is not in the KB. It only gets the final fact AllTestsDone(M1).
6. **Reproduction rate is not used.** It is stored (for example 5 out of 10) but no rule uses it, although it helps the developer in the analysis.
7. **Closed world.** If somebody forgets to enter a fact like BlocksDailyUse, the KB takes it as false and may send an important defect to MR phase.

B.5 Reflect (1 mark)

Practical constraints on deployment

1. **Evolving business rules.** The rules change with every project, because the PL gives new guidelines. Somebody has to keep the rules and the sync-up facts updated, otherwise the KB goes wrong and nobody notices.
2. **Incomplete knowledge.** The data is not complete or the same for everybody. Testers fill severity, module and reproduction details in their own way, which is why the MO has to review every report today.
3. **Privacy.** Defect logs have information about unreleased phones. Such a tool must run inside the company network with the same access control as the portal.
4. **Scalability.** A model project has thousands of defects and many projects run together. The facts must come from the portal automatically, typing them like I did here is not possible.
5. **People.** Testers and developers may not accept a tool which decides the severity or marks a defect as concept. The tool can only support, the responsibility must stay with people.
6. **Technology.** Simple if-then rules cannot handle uncertainty like "probably reproducible", and they cannot handle time.

The single most important improvement

Add a review rule: a severity A defect which the developer marks as concept, limitation or irreproducible must go to the test manager and the MO before it can be closed.

In logic it is ∀d ∀o [Sev(d, A) ∧ Outcome(d, o) ∧ NoCodeChange(o) → NeedsManagerReview(d)], and R13 should fire only after the review is done.

I chose this because it covers the most costly failure in my work. When a developer analyses wrongly and marks a fixable critical issue as concept, no new software is made, the defect is closed on the same binary and the problem can reach the market. More rules or better data would not catch this, because the KB trusts the developer's outcome. The rule is small, the developer still takes the decision, and the reviewer can see from the KB steps why the defect was going to be closed.

What I learned

1. Writing the process as logic forced me to define things exactly. "Blocks daily use" and "blocks readiness" had to become clear conditions instead of a feeling.
2. It showed gaps in the process, such as a returned severity A defect which does not stop the readiness approval in my KB, and sync-up decisions which live only in a meeting.
3. The KB is good for fixed decisions (what is reported, where it goes, when it closes) but it cannot replace the judgement of the tester, the developer and the MO.

Appendix A: Python implementation

I coded the knowledge base in Python in a Jupyter notebook, using only the standard library (textwrap). The facts are tuples. Every rule has conditions, not-conditions and a result. A forward chaining function applies the rules in the three stages of B.3.4 till no new fact comes out, and it saves which rule and which facts gave each new fact, so every answer can be explained. The run started with 61 facts, derived 34 new facts and ended with 95. The screenshots below are from Jupyter Notebook, every code cell with its output below it, in the same order as in the notebook.



Q2: Knowledge base for defect handling in call testing

Name: Neelam Devi
BITS ID: 2025AG05908
Course: AIMLZG557 Artificial and Computational Intelligence

Import

[1]:
```
import textwrap
```

1. Facts

M1 is in PR phase and M2 is in DEV phase. D1 to D9 are made up defects.

[2]:
```
# every fact is a tuple, the first item is the name of the predicate
start_facts = {
    # projects: M1 is in PR phase and M2 is in DEV phase
    ("Project", "M1"), ("Phase", "M1", "PR"), ("AllTestsDone", "M1"),
    ("SyncUpAllows", "M1", "A"),
    ("Project", "M2"), ("Phase", "M2", "Dev"),
    # developer of each module
    ("DeveloperOf", "Call", "DevCall"), ("DeveloperOf", "Audio", "DevAudio"),
    # outcomes which need no new software
    ("NoCodeChange", "Concept"), ("NoCodeChange", "Limitation"),
    ("NoCodeChange", "Irreproducible"),

    # D1: dialer force close on incoming call
    ("Sev", "D1", "A"), ("HasLogs", "D1"), ("FoundIn", "D1", "M1"),
    ("Module", "D1", "Call"), ("Outcome", "D1", "Code"),

    # D2: incoming call needs two swipes to accept, fixed binary released, not seen again
    ("Sev", "D2", "B"), ("BlocksDailyUse", "D2"), ("HasLogs", "D2"),
    ("FoundIn", "D2", "M1"), ("Module", "D2", "Call"), ("Outcome", "D2", "Code"),
    ("BinaryReleased", "D2"), ("NotReproducedOnNew", "D2"),

    # D3: text overlap on the in-call screen
    ("Sev", "D3", "C"), ("HasLogs", "D3"), ("FoundIn", "D3", "M1"), ("Module", "D3", "Call"),

    # D4: one way audio, seen once, wrong module selected
    ("Sev", "D4", "A"), ("SeenOnce", "D4"), ("HasLogs", "D4"), ("FoundIn", "D4", "M1"),
    ("Module", "D4", "Audio"), ("WrongModule", "D4"),

    # D5: call drops on network switch, seen 5 out of 10 times, logs not attached
    ("Sev", "D5", "B"), ("BlocksDailyUse", "D5"), ("ReproRate", "D5", "5of10"),
    ("FoundIn", "D5", "M1"), ("Module", "D5", "Call"),

    # D6: hold icon cropped, seen once, developer marked it as concept
    ("Sev", "D6", "C"), ("SeenOnce", "D6"), ("HasLogs", "D6"), ("FoundIn", "D6", "M2"),
    ("Module", "D6", "Call"), ("Outcome", "D6", "Concept"),

    # D7: crash inside a third party calling app
    ("Sev", "D7", "B"), ("HasLogs", "D7"), ("FoundIn", "D7", "M2"), ("ThirdParty", "D7"),

    # D8: call recording fails, fixed binary released but still seen
    ("Sev", "D8", "B"), ("HasLogs", "D8"), ("FoundIn", "D8", "M2"), ("Module", "D8", "Call"),
    ("Outcome", "D8", "Code"), ("BinaryReleased", "D8"), ("ReproducedOnNew", "D8"),
```

```
    # D9: call mutes after hold, developer could not reproduce it
    ("Sev", "D9", "B"), ("HasLogs", "D9"), ("FoundIn", "D9", "M2"), ("Module", "D9", "Call"),
    ("Outcome", "D9", "Irreproducible"),
}

print(len(start_facts), "facts loaded")
```

```
61 facts loaded
```

2. Rules

R1 to R18 are written as data. Words starting with ? are variables.

[3]:
```
# a rule is (name, conditions, not-conditions, result)
# ?d defect, ?p project, ?s severity, ?m module, ?v developer, ?o outcome
# a not-condition is checked only after the facts it depends on are final,
# so the rules are kept in 3 stages

stage1 = [
    ("R1", [("FoundIn", "?d", "?p"), ("Phase", "?p", "Dev")],
           [], ("Reportable", "?d")),
    ("R2", [("FoundIn", "?d", "?p"), ("Phase", "?p", "PR"),
            ("Sev", "?d", "?s"), ("SyncUpAllows", "?p", "?s")],
           [], ("Reportable", "?d")),
    ("R3", [("FoundIn", "?d", "?p"), ("Phase", "?p", "PR"), ("BlocksDailyUse", "?d")],
           [], ("Reportable", "?d")),
    ("R5", [("Reportable", "?d")], [("HasLogs", "?d")], ("Returned", "?d")),
    ("R6", [("Reportable", "?d"), ("WrongModule", "?d")], [], ("Returned", "?d")),
    ("R7", [("Reportable", "?d"), ("HasLogs", "?d")], [("WrongModule", "?d")],
           ("Submitted", "?d")),
    ("R8", [("Submitted", "?d"), ("Module", "?d", "?m"), ("DeveloperOf", "?m", "?v")],
           [("ThirdParty", "?d")], ("AssignedTo", "?d", "?v")),
    ("R9", [("Submitted", "?d"), ("ThirdParty", "?d")], [], ("ReportToVendor", "?d")),
    ("R10", [("Submitted", "?d"), ("Outcome", "?d", "Code")], [("BinaryReleased", "?d")],
            ("NotRelease", "?d")),
    ("R11", [("Submitted", "?d"), ("Outcome", "?d", "Code"), ("BinaryReleased", "?d")],
            [], ("Release", "?d")),
    ("R12", [("Submitted", "?d"), ("Outcome", "?d", "?o"), ("NoCodeChange", "?o")],
            [], ("ResolveSameBinary", "?d")),
    ("R13", [("ResolveSameBinary", "?d")], [], ("Closed", "?d")),
    ("R14", [("Release", "?d"), ("NotReproducedOnNew", "?d")], [], ("Closed", "?d")),
    ("R15", [("Release", "?d"), ("ReproducedOnNew", "?d")], [], ("Rejected", "?d")),
]

# stage 2: R4 needs the final Reportable facts, R16 needs the final Closed facts
stage2 = [
    ("R4", [("FoundIn", "?d", "?p"), ("Phase", "?p", "PR")], [("Reportable", "?d")],
           ("DeferMR", "?d")),
    ("R16", [("Submitted", "?d"), ("Sev", "?d", "A"), ("FoundIn", "?d", "?p")],
            [("Closed", "?d")], ("BlocksReadiness", "?d", "?p")),
]

# stage 3: R18 needs the final BlocksReadiness facts
stage3 = [
    ("R17", [("Project", "?p"), ("BlocksReadiness", "?d", "?p")], [], ("NotReady", "?p")),
    ("R18", [("Project", "?p"), ("AllTestsDone", "?p")], [("BlocksReadiness", "?d", "?p")],
            ("ReadyForApproval", "?p")),
]

print(len(stage1) + len(stage2) + len(stage3), "rules loaded")
```

```
18 rules loaded
```

3. Forward chaining

Keep applying the rules till no new fact comes out. For every new fact the rule and the facts used are saved in `why`.

[4]:
```
def is_var(x):
    return x.startswith("?")


def match(cond, fact, binding):
    # match one condition with one fact, gives the new binding or None
    if cond[0] != fact[0] or len(cond) != len(fact):
        return None
    b = dict(binding)
    for c, f in zip(cond[1:], fact[1:]):
        if is_var(c):
            if c in b and b[c] != f:
                return None
```

```
            b[c] = f
        elif c != f:
            return None
    return b


def find_all(conds, binding, facts):
    # all the ways in which the conditions can be matched with the facts
    if len(conds) == 0:
        return [binding]
    found = []
    for f in facts:
        b = match(conds[0], f, binding)
        if b is not None:
            found = found + find_all(conds[1:], b, facts)
    return found


def put_values(cond, binding):
    return tuple(binding.get(x, x) for x in cond)


def forward_chain(first_facts):
    facts = set(first_facts)
    why = {}      # new fact -> (rule name, conditions used, not-conditions)
    for stage in [stage1, stage2, stage3]:
        again = True
        while again:
            again = False
            for name, conds, not_conds, result in stage:
                for b in find_all(conds, {}, facts):
                    # closed world: a not-condition fails only if some fact matches it
                    if any(len(find_all([n], b, facts)) > 0 for n in not_conds):
                        continue
                    new = put_values(result, b)
                    if new not in facts:
                        facts.add(new)
                        why[new] = (name, [put_values(c, b) for c in conds],
                                    [put_values(n, b) for n in not_conds])
                        again = True
    return facts, why
```

[5]:
```
facts, why = forward_chain(start_facts)

print("facts given   :", len(start_facts))
print("facts derived :", len(why))
print("facts total   :", len(facts))
```

```
facts given   : 61
facts derived : 34
facts total   : 95
```

4. Functions to print the answers

[6]:
```
def show(f):
    return f[0] + "(" + ", ".join(f[1:]) + ")"


def show_not(n):
    free = [x[1:] for x in n if x.startswith("?")]
    if free:
        # a variable is still free, so it means "no such defect exists"
        return "¬∃" + ", ".join(free) + " " + show(tuple(x.replace("?", "") for x in n))
    return "¬" + show(n)


def show_rule(f, why):
    name, conds, not_conds = why[f]
    parts = [show(c) for c in conds] + [show_not(n) for n in not_conds]
    return name + ": " + " ∧ ".join(parts) + " → " + show(f)


def steps(f, why, done):
    # facts needed to prove f, in the order they were derived
    if f in why and f not in done:
        for c in why[f][1]:
            steps(c, why, done)
        done.append(f)


def ask(goals, facts, why):
```

```
    done = []
    for g in goals:
        if g in facts:
            print(show(g), "?  YES")
            steps(g, why, done)
        else:
            print(show(g), "?  NO, cannot be derived from the KB")
    if done:
        print("  steps:")
    for d in done:
        text = show_rule(d, why)
        print(textwrap.fill(text, 100, initial_indent="    ", subsequent_indent="        "))
```

[7]:
```
# all derived facts with the rule which gave them
for f in sorted(why, key=lambda x: (int(why[x][0][1:]), x)):
    print(why[f][0].ljust(4), show(f))
```

```
R1   Reportable(D6)
R1   Reportable(D7)
R1   Reportable(D8)
R1   Reportable(D9)
R2   Reportable(D1)
R2   Reportable(D4)
R3   Reportable(D2)
R3   Reportable(D5)
R4   DeferMR(D3)
R5   Returned(D5)
R6   Returned(D4)
R7   Submitted(D1)
R7   Submitted(D2)
R7   Submitted(D6)
R7   Submitted(D7)
R7   Submitted(D8)
R7   Submitted(D9)
R8   AssignedTo(D1, DevCall)
R8   AssignedTo(D2, DevCall)
R8   AssignedTo(D6, DevCall)
R8   AssignedTo(D8, DevCall)
R8   AssignedTo(D9, DevCall)
R9   ReportToVendor(D7)
R10  NotRelease(D1)
R11  Release(D2)
R11  Release(D8)
R12  ResolveSameBinary(D6)
R12  ResolveSameBinary(D9)
R13  Closed(D6)
R13  Closed(D9)
R14  Closed(D2)
R15  Rejected(D8)
R16  BlocksReadiness(D1, M1)
R17  NotReady(M1)
```

5. Queries

[8]:
```
print("Query 1: D3 is a display overlap (severity C) in project M1, which is in PR phase.")
print("         Should it be reported now?")
print()
ask([("Reportable", "D3"), ("DeferMR", "D3")], facts, why)
print()
if ("DeferMR", "D3") in facts:
    print("Answer: D3 is not reported now, it goes to MR phase")
```

```
Query 1: D3 is a display overlap (severity C) in project M1, which is in PR phase.
         Should it be reported now?

Reportable(D3) ?  NO, cannot be derived from the KB
DeferMR(D3) ?  YES
  steps:
    R4: FoundIn(D3, M1) ∧ Phase(M1, PR) ∧ ¬Reportable(D3) → DeferMR(D3)

Answer: D3 is not reported now, it goes to MR phase
```

[9]:
```
print("Query 2: What happens to D1, the dialer force close (severity A)?")
print()
ask([("Reportable", "D1"), ("Submitted", "D1"), ("AssignedTo", "D1", "DevCall"),
     ("NotRelease", "D1")], facts, why)
print()
print("Answer: D1 is submitted, assigned to DevCall and waits in not release state")
```

```
Query 2: What happens to D1, the dialer force close (severity A)?

Reportable(D1) ?  YES
Submitted(D1) ?  YES
AssignedTo(D1, DevCall) ?  YES
NotRelease(D1) ?  YES
  steps:
    R2: FoundIn(D1, M1) ∧ Phase(M1, PR) ∧ Sev(D1, A) ∧ SyncUpAllows(M1, A) → Reportable(D1)
    R7: Reportable(D1) ∧ HasLogs(D1) ∧ ¬WrongModule(D1) → Submitted(D1)
    R8: Submitted(D1) ∧ Module(D1, Call) ∧ DeveloperOf(Call, DevCall) ∧ ¬ThirdParty(D1) →
        AssignedTo(D1, DevCall)
    R10: Submitted(D1) ∧ Outcome(D1, Code) ∧ ¬BinaryReleased(D1) → NotRelease(D1)

Answer: D1 is submitted, assigned to DevCall and waits in not release state
```

[10]:
```
print("Query 3: D4 is one way audio (severity A, seen once) and the module selected is wrong.")
print("         Does it reach the developer?")
print()
ask([("Reportable", "D4"), ("Returned", "D4"), ("Submitted", "D4")], facts, why)
print()
if ("Returned", "D4") in facts and ("Submitted", "D4") not in facts:
    print("Answer: D4 is reportable but the MO returns it to the tester, it is not submitted")
```

```
Query 3: D4 is one way audio (severity A, seen once) and the module selected is wrong.
         Does it reach the developer?

Reportable(D4) ?  YES
Returned(D4) ?  YES
Submitted(D4) ?  NO, cannot be derived from the KB
  steps:
    R2: FoundIn(D4, M1) ∧ Phase(M1, PR) ∧ Sev(D4, A) ∧ SyncUpAllows(M1, A) → Reportable(D4)
    R6: Reportable(D4) ∧ WrongModule(D4) → Returned(D4)

Answer: D4 is reportable but the MO returns it to the tester, it is not submitted
```

[11]:
```
print("Query 4: New software is released. What happens to D2 and D8?")
print()
ask([("Closed", "D2"), ("Rejected", "D8")], facts, why)
print()
if ("Closed", "D2") in facts:
    print("Answer: D2 is closed by the tester")
if ("Rejected", "D8") in facts:
    print("        D8 is rejected and goes back to the developer")
```

```
Query 4: New software is released. What happens to D2 and D8?

Closed(D2) ?  YES
Rejected(D8) ?  YES
  steps:
    R3: FoundIn(D2, M1) ∧ Phase(M1, PR) ∧ BlocksDailyUse(D2) → Reportable(D2)
    R7: Reportable(D2) ∧ HasLogs(D2) ∧ ¬WrongModule(D2) → Submitted(D2)
    R11: Submitted(D2) ∧ Outcome(D2, Code) ∧ BinaryReleased(D2) → Release(D2)
    R14: Release(D2) ∧ NotReproducedOnNew(D2) → Closed(D2)
    R1: FoundIn(D8, M2) ∧ Phase(M2, Dev) → Reportable(D8)
    R7: Reportable(D8) ∧ HasLogs(D8) ∧ ¬WrongModule(D8) → Submitted(D8)
    R11: Submitted(D8) ∧ Outcome(D8, Code) ∧ BinaryReleased(D8) → Release(D8)
    R15: Release(D8) ∧ ReproducedOnNew(D8) → Rejected(D8)

Answer: D2 is closed by the tester
        D8 is rejected and goes back to the developer
```

[12]:
```
print("Query 5: Can the MO give readiness approval for project M1?")
print()
ask([("ReadyForApproval", "M1"), ("NotReady", "M1")], facts, why)
print()
if ("NotReady", "M1") in facts:
    blockers = sorted(f[1] for f in facts if f[0] == "BlocksReadiness" and f[2] == "M1")
    print("Answer: no, M1 is not ready")
    print("        defects blocking M1:", blockers)
```

```
Query 5: Can the MO give readiness approval for project M1?

ReadyForApproval(M1) ?  NO, cannot be derived from the KB
NotReady(M1) ?  YES
  steps:
    R2: FoundIn(D1, M1) ∧ Phase(M1, PR) ∧ Sev(D1, A) ∧ SyncUpAllows(M1, A) → Reportable(D1)
    R7: Reportable(D1) ∧ HasLogs(D1) ∧ ¬WrongModule(D1) → Submitted(D1)
    R16: Submitted(D1) ∧ Sev(D1, A) ∧ FoundIn(D1, M1) ∧ ¬Closed(D1) → BlocksReadiness(D1, M1)
    R17: Project(M1) ∧ BlocksReadiness(D1, M1) → NotReady(M1)

Answer: no, M1 is not ready
        defects blocking M1: ['D1']
```

Other defects and a what-if case

[13]:
```
print("Other defects, to check the remaining rules")
print()
ask([("Returned", "D5"), ("Closed", "D6"), ("ReportToVendor", "D7"), ("Closed", "D9")],
    facts, why)
```

```
Other defects, to check the remaining rules

Returned(D5) ?  YES
Closed(D6) ?  YES
ReportToVendor(D7) ?  YES
Closed(D9) ?  YES
  steps:
    R3: FoundIn(D5, M1) ∧ Phase(M1, PR) ∧ BlocksDailyUse(D5) → Reportable(D5)
    R5: Reportable(D5) ∧ ¬HasLogs(D5) → Returned(D5)
    R1: FoundIn(D6, M2) ∧ Phase(M2, Dev) → Reportable(D6)
    R7: Reportable(D6) ∧ HasLogs(D6) ∧ ¬WrongModule(D6) → Submitted(D6)
    R12: Submitted(D6) ∧ Outcome(D6, Concept) ∧ NoCodeChange(Concept) → ResolveSameBinary(D6)
    R13: ResolveSameBinary(D6) → Closed(D6)
    R1: FoundIn(D7, M2) ∧ Phase(M2, Dev) → Reportable(D7)
    R7: Reportable(D7) ∧ HasLogs(D7) ∧ ¬WrongModule(D7) → Submitted(D7)
    R9: Submitted(D7) ∧ ThirdParty(D7) → ReportToVendor(D7)
    R1: FoundIn(D9, M2) ∧ Phase(M2, Dev) → Reportable(D9)
    R7: Reportable(D9) ∧ HasLogs(D9) ∧ ¬WrongModule(D9) → Submitted(D9)
    R12: Submitted(D9) ∧ Outcome(D9, Irreproducible) ∧ NoCodeChange(Irreproducible) →
        ResolveSameBinary(D9)
    R13: ResolveSameBinary(D9) → Closed(D9)
```

[14]:
```
print("What if the fixed binary of D1 is released and D1 is not seen on it?")
print()
new_start = start_facts | {("BinaryReleased", "D1"), ("NotReproducedOnNew", "D1")}
facts2, why2 = forward_chain(new_start)
ask([("Closed", "D1"), ("ReadyForApproval", "M1"), ("NotReady", "M1")], facts2, why2)
```

```
What if the fixed binary of D1 is released and D1 is not seen on it?

Closed(D1) ?  YES
ReadyForApproval(M1) ?  YES
NotReady(M1) ?  NO, cannot be derived from the KB
  steps:
    R2: FoundIn(D1, M1) ∧ Phase(M1, PR) ∧ Sev(D1, A) ∧ SyncUpAllows(M1, A) → Reportable(D1)
    R7: Reportable(D1) ∧ HasLogs(D1) ∧ ¬WrongModule(D1) → Submitted(D1)
    R11: Submitted(D1) ∧ Outcome(D1, Code) ∧ BinaryReleased(D1) → Release(D1)
    R14: Release(D1) ∧ NotReproducedOnNew(D1) → Closed(D1)
    R18: Project(M1) ∧ AllTestsDone(M1) ∧ ¬∃d BlocksReadiness(d, M1) → ReadyForApproval(M1)
```

The notebook output matches the answers worked out in B.3.5. It also checks D5 (returned, no logs), D6 and D9 (closed on the same binary), D7 (goes to the app vendor) and the what-if case of Query 5.